Homework 3
Indexed Paged List Implementation

COMS 2280 Fall 2025

Due at **11:59 PM, Nov. 19th**

Contents

1 Homework Overview **2**

2 Core Concepts **2**
- 2.1 Paged List Structure . . . 2
- 2.2 The Page Index (Fast-Lane) . . . 2
- 2.3 Performance Characteristics . . . 2
- 2.4 Page Management . . . 2

3 Assignment Tasks **3**
- 3.1 Importing the Skeletal Code . . . 3
- 3.2 To Do . . . 3

4 Class and Method Descriptions **3**
- 4.1 `IndexedPagedList<E>` . . . 3
- 4.2 `PagedListIterator` . . . 4

5 Debugging Output **4**

6 Testing **5**

7 Submission **5**
- 7.1 Deliverables . . . 5
- 7.2 Important Instructions . . . 5
- 7.3 Significant Dates . . . 5

8 Working Example **6**

1 Homework Overview

This homework is designed to provide hands-on experience with implementing a complex, custom data structure that balances different performance trade-offs. You will implement the `IndexedPagedList<E>` class, a `List` implementation that uses a doubly-linked list of "pages" (nodes), where each page stores multiple items in an array.

The core challenge is to maintain a separate "fast-lane" index (an `ArrayList`) that allows for fast, $O(\log N_{\text{pages}})$, index-based access, while also managing the list's structure during modifications. These modifications (like `add` and `remove`) will require page splits and merges, resulting in an $O(N_{\text{pages}})$ performance.

2 Core Concepts

The `IndexedPagedList` combines the strengths of an array (fast indexing) and a linked list (efficient insertions/deletions in the middle, in terms of data movement). It does this by creating a two-level structure.

2.1 Paged List Structure

The primary structure is a doubly-linked list of `Page<E>` nodes.

- **Head and Tail Dummies:** The list is anchored by `head` and `tail` dummy nodes, just like a standard `LinkedList`.
- **Pages as Nodes:** Each `Page<E>` node is not a single element. Instead, it holds an array of elements (e.g., `E[] items`).
- **Page Capacity:** Each page's array has a fixed capacity, defined by the constant `PAGE_CAPACITY`. A page also tracks its current `count` of items.

2.2 The Page Index (Fast-Lane)

To avoid $O(N)$ traversal for `get(int index)`, the list maintains a separate `ArrayList<IndexEntry<E> >` called `pageIndex`.

- This `pageIndex` acts as a top-level directory. It stores one `IndexEntry` for **every** `Page` in the linked list.
- Each `IndexEntry` stores three key pieces of information:
  1. A direct reference to its corresponding `Page<E>` object.
  2. The **logical index** of the **first** item on that page.
  3. The current **item count** on that page.
- **Example:** If `PAGE_CAPACITY` is 4, a list with 10 items might look like this:
  - **Page 0:** (Items: A, B, C, D) → **IndexEntry 0:** (Page: Page0, Index: 0, Count: 4)
  - **Page 1:** (Items: E, F, G, H) → **IndexEntry 1:** (Page: Page1, Index: 4, Count: 4)
  - **Page 2:** (Items: I, J, -, -) → **IndexEntry 2:** (Page: Page2, Index: 8, Count: 2)

2.3 Performance Characteristics

- **Index-based Access (`get`, `set`, `listIterator(pos)`):** These operations are fast, $O(\log N_{\text{pages}})$. They work by performing a binary search on `pageIndex` to find the correct `IndexEntry`, which points directly to the correct `Page`.
- **Modifications (`add(int, E)`, `remove(int)`):** These operations are $O(N_{\text{pages}})$.
  - Finding the target page is fast ($O(\log N_{\text{pages}})$).
  - However, after an insertion or deletion, the `logicalIndex` of **all subsequent** `IndexEntry` objects in the `pageIndex` must be updated (e.g., incremented by 1). This update requires a linear scan of the `pageIndex`, making the whole operation $O(N_{\text{pages}})$. This is still much faster than a standard `LinkedList`'s $O(N)$ modification time.

2.4 Page Management

To keep the structure efficient, you must manage the fill-level of the pages.

- **Page Split:** When `add` is called on a page that is already full (`count == PAGE_CAPACITY`), you must

perform a **split**. This involves creating a new page, moving the second half of the items from the full page to the new page, and inserting the new page (and its corresponding `IndexEntry`) into the list and index.

- **Page Merge/Rebalance:** When `remove` causes a page's count to drop below `HALF_CAPACITY`, you must perform a **rebalance** or **merge** with its successor (next) page.
  - **Rebalance:** If the successor page has items to spare (`count > HALF_CAPACITY`), move one item from the successor to the current page.
  - **Merge:** If the successor is also at or below half capacity, move all items from the successor into the current page and remove the (now empty) successor page and its `IndexEntry`.

3 Assignment Tasks

3.1 Importing the Skeletal Code

You are given skeletal code containing the following file:

- `IndexedPagedList.java`

Import this homework into your IDE of choice. This class extends `AbstractSequentialList`, which provides implementations for many `List` methods (like `get` and `set`) by using the `listIterator()`.

3.2 To Do

- Review the provided skeletal code, and implement all incomplete functions or methods.
- Implement the constructor `IndexedPagedList()`.
- Implement the core public methods: `add(int pos, E item)`, `remove(int pos)`, `size()`, `add(E item)`, and `listIterator(int pos)`.
- Implement the private helper methods: `findPageForLogicalIndex(int)`, `findIndexInPageIndex(Page<E>)`, and `updatePageIndex(int, int)`.
- Implement the entire `private class PagedListIterator`. This is a major part of the assignment and includes its constructor and all its methods (`next`, `previous`, `add`, `remove`, `set`, etc.).
- You are provided with the complete helper inner classes `Page<E>`, `IndexEntry<E>`, and `PageInfo<T>`. You are also provided with two `toStringInternal()` debugging methods. Do not modify these provided classes or methods.
- Refer to the class descriptions in Section 4 to understand the method signatures and their intended use.
- Every method and field described in the skeleton needs to be implemented correctly to score full points.

4 Class and Method Descriptions

The project consists of the `IndexedPagedList` and its private inner classes.

4.1 `IndexedPagedList<E>`

This is the main class you will implement. It extends `AbstractSequentialList<E>`.

- `IndexedPagedList()`: The constructor must initialize the `head` and `tail` dummy nodes, link them, and initialize the `pageIndex` `ArrayList`.
- `add(int pos, E item)`: This is the core insertion logic. You must:
  1. Handle all exceptions (null item, index out of bounds).
  2. Handle the special case of an empty list (create the first page).
  3. Find the correct page and offset using `findPageForLogicalIndex`.
  4. If the page has space (`count < PAGE_CAPACITY`):
     - Add the item to the page's array.
     - Update the page's `count` in its `IndexEntry`.
     - Call `updatePageIndex` to increment the `logicalIndex` of all subsequent entries.

5. If the page is full:
   - Perform a **split**. Create a new `Page` and link it after the current one.
   - Move `HALF_CAPACITY` items from the end of the current page to the new page.
   - Update the counts for both pages in the `pageIndex`.
   - Create a new `IndexEntry` for the new page and add it to `pageIndex` at the correct position.
   - Determine which page (old or new) the `item` should be added to based on the original `pos`, and add it.
   - Call `updatePageIndex` to update all subsequent entries.

- `remove(int pos)`: This is the core removal logic. You must:
  1. Find the page and offset for the item to remove.
  2. Remove the item from the page's array (shifting elements left).
  3. Update the page's `count` in its `IndexEntry`.
  4. Check for underflow (`count < HALF_CAPACITY`). If it occurs:
     - Check the successor page (`currentPage.next`).
     - **Rebalance:** If `successor.count > HALF_CAPACITY`, append the first item from the successor to the current page. Update counts and logical indices in the `pageIndex` for both pages.
     - **Merge:** If `successor.count <= HALF_CAPACITY`, move all items from the successor to the current page. Unlink and remove the successor page. Remove the successor's `IndexEntry` from `pageIndex`.
  5. Handle the edge case of removing the last item from the list, which should remove the last page.
  6. Call `updatePageIndex` to decrement the `logicalIndex` of all subsequent entries.
- `listIterator(int pos)`: Must return a new `PagedListIterator` instance. The iterator's constructor must be efficient ($O(\log N_{\text{pages}})$), using `findPageForLogicalIndex` to immediately seek to the correct starting page and offset.

4.2 `PagedListIterator`

This private inner class implements `ListIterator<E>`.

- **Constructor**: As stated above, must use `findPageForLogicalIndex` to find the starting `currentPage` and `pageOffset` for the given `pos`.
- `hasNext()` / `hasPrevious()`: Check if the `logicalIndex` is within the bounds of `totalSize`.
- `next()` / `previous()`: These must correctly return the next/previous item and advance/retreat the iterator's state (`logicalIndex`, `pageOffset`). This logic **must handle crossing page boundaries** (i.e., when `pageOffset` reaches 0 or `currentPage.count`).
- `add(E item)` / `remove()`: These methods **must call the outer class's** `IndexedPagedList.this.add (logicalIndex, item)` and `IndexedPagedList.this.remove (lastItemIndex)` methods. They should **not** modify the pages directly. After calling the outer method, they must update their own state (`logicalIndex`, `expectedModificationCount`) and re-synchronize their physical position (`currentPage`, `pageOffset`) by calling `findPageForLogicalIndex`.
- `set(E item)`: This method can modify the page's array directly, as it does not cause a structural modification. It must find the correct page/offset for the `lastItemIndex`.
- **Fail-Fast:** The iterator must be fail-fast. You must use the `modificationCount` and `expectedModificationCou` fields to throw a `ConcurrentModificationException` if the list is modified externally.

5 Debugging Output

The skeleton code provides a `toStringInternal()` method to help you visualize the state of your list. It prints the contents of each page, with `-` representing an empty slot in the page's array. You are encouraged to use this in your `main` method or JUnit tests to verify your logic.

```
Page Capacity: 4
```

```
An empty list: []

After adding 1, 2, 3:
[(1, 2, 3, -)]

After adding 4:
[(1, 2, 3, 4)]

After adding 5 (at index 4), triggering a split:
[(1, 2, -, -), (3, 4, 5, -)]

After adding 8 (at index 2), to the first page:
[(1, 2, 8, -), (3, 4, 5, -)]
```

6 Testing

A comprehensive suite of unit tests (using JUnit 5) is crucial for verifying the correctness of this complex data structure. **You are not required to submit your tests** for this homework.

You are strongly encouraged to write your own tests and share them with your peers. Test all edge cases including but not limited to:

- Test on an empty list (`size()`, `add()`, `iterator()`).
- Test adding to trigger a **page split**.
- Test adding to the beginning, middle, and end of the list.
- Test removing to trigger a **rebalance**.
- Test removing to trigger a **full merge**.

7 Submission

Write your classes in the `edu.iastate.cs2280.hw3` package.

7.1 Deliverables

- A zip file containing all the Java source files. Please do not turn in any generated `.class` files.
- The zip should include the `src` folder and `pom.xml` at its root.
- The zip file should be named `FirstName_LastName.zip`.
- Make sure the zip file you are uploading is the correct version and not empty.

7.2 Important Instructions

1. Write Javadoc for documenting your homework and provide additional comments where necessary.
2. Do not change any function signatures, class names, or provided helper classes from the skeleton. Any violation will be penalized by up to a 50% reduction in score.
3. For this homework, all the private method we have specified cannot be changed and will be considered contract violation.
4. You are welcome to add additional private helper methods and fields.
5. If the project is not passing the Gradescope Autograder, then recheck your submission to make sure there are no compilation errors or contract violations. Contract violations can include wrong package name, folder structures on top of modifying function signatures.

7.3 Significant Dates

The homework is due on **Nov. 19th at midnight**. You can utilize a 3-day extension without any penalty or permission. The homework may be marked as late but will not be penalized up to Nov. 22nd midnight. The final day to submit the homework is Nov. 23rd, which will incur a 10% penalty. No further extensions will be allowed.

You can also accrue 10% bonus points by submitting early. For each day you submit early, limited to five

days (from Nov. 14th), you get a 2% bonus point. These points are calculated based on your homework's graded score, not the total possible points.

8 Working Example

Below is a sample `main` method **(not part of the homework or submission)** and its corresponding output using `toStringInternal()`. This demonstrates a more complex sequence of operations, including page splits, a rebalance, a merge, and iterator modifications.

```java
public class Main {
  public static void main(String[] args) {
    // Note: PAGE_CAPACITY is 4 in the skeleton

    IndexedPagedList<String> list = new IndexedPagedList<>();
    System.out.println("New list:   " + list.toStringInternal());
    list.add("A");
    list.add("B");
    list.add("C");
    list.add("D");
    System.out.println("Add A,B,C,D:  "
        + list.toStringInternal() + " (Size: " + list.size() + ")");

    // --- Test Split ---
    list.add("E");
    System.out.println("Add E (split):"
        + list.toStringInternal() + " (Size: " + list.size() + ")");
    list.add("F");
    System.out.println("Add F:        "
        + list.toStringInternal() + " (Size: " + list.size() + ")");
    list.add("G");
    System.out.println("Add G (split):"
        + list.toStringInternal() + " (Size: " + list.size() + ")");

    // --- Test Rebalance ---
    list.remove(2);
    System.out.println("Remove C @ 2: "
        + list.toStringInternal() + " (Size: " + list.size() + ")");
    System.out.println("(Rebalance triggered)");

    // --- Test Merge ---
    list.remove(2);
    System.out.println("Remove D @ 2: "
        + list.toStringInternal() + " (Size: " + list.size() + ")");
    System.out.println("(Merge triggered)");

    // --- Test Iterator ---
    ListIterator<String> iter = list.listIterator(3);
    System.out.println("\nIterator @ 3: " + list.toStringInternal(iter));
    System.out.println("iter.next() -> " + iter.next());
    System.out.println("After next(): " + list.toStringInternal(iter));
    System.out.println("iter.previous() -> " + iter.previous());
    System.out.println("After prev(): " + list.toStringInternal(iter));
    iter.remove();
    System.out.println("iter.remove():"
        + list.toStringInternal(iter) + " (Size: " + list.size() + ")");
    iter.add("Z");
    System.out.println("iter.add(Z):  "
        + list.toStringInternal(iter) + " (Size: " + list.size() + ")");
    System.out.println("iter.next() -> " + iter.next());
    System.out.println("Final state:  "
        + list.toStringInternal(iter) + " (Size: " + list.size() + ")");
  }
}
```

Listing 1: Sample Main Method for Debugging

```
(Expected Output from a correct implementation)

New list:   []
Add A,B,C,D:  [(A, B, C, D)] (Size: 4)
Add E (split):[(A, B, -, -), (C, D, E, -)] (Size: 5)
Add F:        [(A, B, -, -), (C, D, E, F)] (Size: 6)
Add G (split):[(A, B, -, -), (C, D, -, -), (E, F, G, -)] (Size: 7)
Remove C @ 2: [(A, B, -, -), (D, E, -, -), (F, G, -, -)] (Size: 6)
(Rebalance triggered)
Remove D @ 2: [(A, B, -, -), (E, F, G, -)] (Size: 5)
(Merge triggered)

Iterator @ 3: [(A, B, -, -), (E, | F, G, -)]
iter.next() -> F
After next(): [(A, B, -, -), (E, F, | G, -)]
iter.previous() -> F
After prev(): [(A, B, -, -), (E, | F, G, -)]
iter.remove():[(A, B, -, -), (E, | G, -, -)] (Size: 4)
iter.add(Z):  [(A, B, -, -), (E, Z, | G, -)] (Size: 5)
iter.next() -> G
Final state:  [(A, B, -, -), (E, Z, G, | -)] (Size: 5)
```

Please note that this project doesn't have or require a `main()` method. However, you are encouraged to use the above to test and debug your code.